Beyond the Call Graph: Infrastructure-Aware Code Graphs, Formal Concept Lattices, and Executable Drift Contracts for Coding Agents

LEIO Research Group
Autonomous Software Engineering & Systems Verification Lab
`leio-code v2.6.5`

October 2026

Abstract

Repository-level coding agents spend 60–80% of their interaction turns and token budgets executing iterative lexical searches (`grep`/`rg`) and partial file reads to reconstruct software architecture. Existing repository indexing tools—such as AST symbol maps (RepoMap) and Language Server Protocol (LSP) call graphs—capture intra-language syntax but are structurally blind to *operational topology*: environment variable contracts across code and deployment manifests, Redis key namespace lifecycles, HTTP/MCP route tables, Docker and systemd service wiring, and cross-language subprocess boundaries. Furthermore, when combining lexical, structural, and configuration retrieval signals, we show that naive multi-channel Reciprocal Rank Fusion (RRF) suffers from *single-token channel amplification*, where monolithic "god files" and test fixtures outrank cohesive implementation modules.

We present LEIO Code, a deterministic, sub-second repository intelligence engine implemented in Rust that unifies AST call/import graphs, operational infrastructure topology, Formal Concept Analysis (FCA) Galois lattices, fail-closed Oxigraph SPARQL grounding, and 102 executable architectural drift contracts (*doctors*) behind a versioned JSON and Model Context Protocol (MCP) surface. To rank task-relevant context bundles accurately in a single call, we introduce *Coverage-Calibrated Multi-Faceted RRF* augmented with an evolving Git co-edit playbook (ACE). Evaluated on our 349-file polyglot repository (8,343 formal objects, 4,061 attributes, 67,642 structural incidences, and 2,474 closed Galois concepts induced in 593 ms), LEIO Code achieves MRR = 0.9800, Hit@1 = 96.0%, and Hit@3 = 100.0% across a 25-task multi-category benchmark, outperforming lexical `ripgrep` (MRR = 0.3193, Hit@1 = 20.0%), AST-only RepoMap (MRR = 0.7367, Hit@1 = 68.0%), and naive 3-layer RRF (MRR = 0.6847, Hit@1 = 52.0%), while reducing localization token consumption by 29.4× (17,261 → 587 tokens) and agent tool calls by 3.45×. On coupled cross-layer edits, the ACE co-edit playbook lifts unindexed manifest/documentation partner recall (Recall@6) from 16.7% to 100.0%.

1 Introduction

Modern autonomous coding agents (e.g., Claude Code, Codex, Cursor, Gemini Antigravity, SWE-agent [1]) operate in an iterative tool-use loop over a target software repository. Despite rapid advances in underlying foundation models, empirical analyses of repository-level benchmarks [2, 3] reveal a persistent bottleneck: agents spend the majority of their tokens and wall-clock time on *localization and orientation*. When issued a maintenance or debugging task, an agent typically invokes dozens of lexical `ripgrep` (`rg`) queries, inspects noisy keyword hits in documentation or test fixtures, and manually traces imports across files.

To mitigate lexical thrashing, recent systems provide agents with AST-derived repository maps (e.g., Aider's Tree-sitter PageRank RepoMap [4], LocAgent [5], RepoGraph [6], and CodexGraph [7]). While effective for pure intra-language symbol navigation, AST-only graphs fail on production systems engineering for four fundamental reasons:

1. **Operational Topology Blindness:** Real-world failures frequently cross the boundary between application source code and operational infrastructure—such as an environment variable read via `std::env::var` in Rust that is missing from a `docker-compose.yml` or Cloud Run deployment script, a Redis key namespace written without a TTL ('SET' without 'EX'), or a Node.js MCP wrapper spawning a Rust binary across a subprocess boundary. Pure AST/LSP graphs have no vertices or edges for these operational artifacts.
2. **Single-Token RRF Pollution in Multi-Layer Retrieval:** Combining path matching, AST symbol indexing, and operational config indexing via standard Reciprocal Rank Fusion (RRF) [8] unexpectedly *degrades* top-1 precision (Hit@1 drops from 68.0% to 52.0% in our ablation study). Because RRF rewards each active ranked list with $\frac{1}{k_0+r}$, a monolithic "god file" or integration test that matches a single generic query token across all three channels outranks a focused production module that matches five distinct query tokens within the symbol channel.
3. **Absence of Formal Taxonomic and Ontological Grounding:** Vector rag over markdown documentation returns plausible text snippets even when no authoritative fact exists in the repository, inducing agent hallucinations on architectural invariants.
4. **Passive vs. Executable Architecture Contracts:** Architectural rules written in prose (`AGENTS.md`, `README.md`) decay silently unless paired with deterministic, structural static analyzers that verify cross-layer coherence on every edit.

We address these challenges with LEIO Code ('v2.6.5'), an infrastructure-aware repository intelligence and formal verification substrate for coding agents. This paper makes five contributions:

- **Heterogeneous Operational Code Graph (§2):** We formalize and implement a polyglot repository index (`RepoIndex`) spanning 11 vertex kinds and 12 cross-domain edge relations across Rust, Python, TypeScript/JavaScript, Go, Shell, Dockerfiles, Docker Compose, systemd units, and JSON/YAML schemas.
- **In-Process Formal Concept Lattice & Fail-Closed SPARQL (§3):** We map the repository's structural incidence matrix (G, M, I) into a Galois concept lattice $\mathfrak{B}(G, M, I)$ via a native Rust `In-Close` bitset engine, verify category-theoretic functor coherence ($\mathcal{C}_{\text{functor}} = 1.000$) between wiki documentation hierarchies and code concepts, export an OWL TBox, and enforce fail-closed SPARQL grounding (`leio-knowledge-core`) that refuses to answer when variable bindings are empty.
- **Coverage-Calibrated Multi-Faceted RRF & ACE Playbook (§4):** We diagnose the single-token RRF amplification pathology and introduce a coverage-calibrated scoring formulation combining multi-symbol token union, BM25-style symbol-count damping, intent-gated config channels, production role priors, and an incremental Git co-edit playbook (`Playbook`).
- **Executable Drift Contracts (§5):** We ship 102 deterministic structural analyzers (*doctors*) paired with commit-streak warning triage that detect cross-language and operational drift before tests run.

- **Rigorous Empirical & Ablation Evaluation (§6):** Across a 25-task multi-category benchmark and a 7-task golden retrieval suite, LEIO Code achieves 0.9800 and 1.0000 MRR respectively, cutting localization tokens by 29.4×.

2 Heterogeneous Infrastructure-Aware Code Graph

Definition 1 (Heterogeneous Operational Repository Graph)**.** *A repository index is a typed, directed multigraph* $\mathcal{G} = (V, E, \tau_V, \tau_E, \lambda)$ *where each vertex* $v \in V$ *has kind* $\tau_V(v) \in \mathcal{K}_V$*, each directed edge* $e = (u, v) \in E$ *has relation type* $\tau_E(e) \in \mathcal{K}_E$*, and* $\lambda(v)$ *binds* v *to a deterministic source span* $(path, line_{start}, line_{end})$*.*

Unlike AST-only graphs where $\mathcal{K}_V = \{\texttt{File}, \texttt{Symbol}\}$, LEIO Code defines 11 first-class vertex kinds partitioning syntax, runtime state, and deployment topology:

$$\mathcal{K}_V = \{\texttt{File}, \texttt{Symbol}, \texttt{Import}, \texttt{EnvVar}, \texttt{RedisKey}, \texttt{ApiRoute}, \texttt{DockerService}, \texttt{DeployTarget}, \texttt{Cartridge}, \texttt{SubprocessCaller}, \texttt{Binary}, \texttt{WikiSection}\}.$$

Edges $\mathcal{K}_E$ capture both intra-language syntax and cross-layer operational couplings:

- **Syntactic Call & Import Resolution:** `Calls`, `CalledBy`, `Imports`, and workspace-resolved module edges `ResolvedImports`/`ResolvedImportedBy` (resolving Rust `crate::`/`super::`/`mod`, TypeScript/JS relative paths and `package.json` workspaces, and Python package paths).
- **Environment Contract Edges:** `ReadsEnv`(u, e) when code symbol u reads environment variable $e \in V_{\texttt{EnvVar}}$ (e.g., `std::env::var`, `os.getenv`, `process.env`), and `DeclaresEnv`(s, e) when a Docker Compose service, Dockerfile, systemd unit, or '.env.example' declares e.
- **Stateful Keyspace Edges:** `ReadsRedisKey`, `WritesRedisKey`, and `SetsTtl` extracted from Redis command invocations (`GET`, `SET`, `SETEX`, `EXPIRE`, `HSET`, `DEL`), recording key prefix templates (e.g., `phone_route:{id}`) and TTL presence.
- **Service, Route, and Subprocess Edges:** `ExposesRoute` (HTTP/axum/FastAPI/Express routes and MCP tool registrations), `SpawnsBinary` (cross-language `Command::new` / `spawnSync` invocations linking JS/Python wrappers to Rust binaries), and `DeploysService`.

3 Formal Concept Lattice, Functor Coherence, and Fail-Closed Grounding

To support taxo-structural navigation and prevent hallucinated architectural explanations, LEIO Code transforms the repository graph $\mathcal{G}$ into a formal concept lattice and an RDF/OWL knowledge base.

3.1 Galois Concept Lattice Induction via Bitset `In-Close`

Definition 2 (Repository Formal Context and Galois Lattice)**.** *Let* G *be the set of repository objects (files, symbols, routes, services, and wiki sections),* M *be the set of structural attributes (e.g.,* `lang:rust`*,* `dir:src/doctors`*,* `calls:run_gate`*,* `reads_env:LEIO_CODE_BIN`*,* `kind:function`*), and* $I \subseteq G \times M$ *be the binary incidence relation. For* $A \subseteq G$ *and* $B \subseteq M$*, the Galois derivation operators are:*

$$A' = \{m \in M \mid \forall g \in A,\ (g, m) \in I\}, \qquad B' = \{g \in G \mid \forall m \in B,\ (g, m) \in I\}. \tag{1}$$

A pair (A, B) *is a* formal concept *iff* $A' = B$ *and* $B' = A$*, where* A *is the* extent *and* B *is the* intent*. The set of all formal concepts* $\mathfrak{B}(G, M, I)$ *forms a complete lattice under the partial order* $(A_1, B_1) \leq (A_2, B_2) \iff A_1 \subseteq A_2 \iff B_2 \subseteq B_1$*.*

In `src/fca.rs`, we implement a dependency-free, in-process Rust `In-Close` induction engine operating over packed 64-bit word bitsets (`BitSet`). Given the canonical sorted incidence pairs I, the engine computes all closed concepts $\mathfrak{B}(G, M, I)$, reduces the subconcept order $\leq$ to its transitive reduction (Hasse cover links $c_1 \prec c_2$), assigns each object $g \in G$ to its most specific primary concept $\mu(g) = (\{g\}'', \{g\}')$, and computes a cryptographic incidence fingerprint:

$$\mathrm{fp}(I) = \texttt{sha256:structural-incidence-v1:} \parallel \mathrm{SHA256}\left(\bigoplus_{(g,m)\in I} |g|_{\mathrm{le64}} \| g \| |m|_{\mathrm{le64}} \| m \right). \tag{2}$$

Because freshness is governed by $\mathrm{fp}(I)$ rather than filesystem timestamps alone, a timestamp-only reindex never invalidates a structurally unchanged concept lattice.

3.2 Category-Theoretic Functor Coherence

We view the Hasse cover graph of $\mathfrak{B}(G, M, I)$ as a finite category $\mathcal{C}_{\mathrm{lat}}$ whose objects are concepts and whose generating morphisms are $c_1 \xrightarrow{\texttt{subClassOf}} c_2$, with composition given by directed reachability. Simultaneously, markdown documentation headings inside the repository form a syntactic category $\mathcal{C}_{\mathrm{wiki}}$ where nested sections satisfy $h_{\mathrm{child}} \xrightarrow{\texttt{subClassOf}} h_{\mathrm{parent}}$. Given an object alignment $F : \mathrm{Obj}(\mathcal{C}_S) \to \mathrm{Obj}(\mathcal{C}_T)$, `src/category.rs` evaluates functorial coherence:

$$\mathrm{Coherence}(F) = \frac{1}{|\mathrm{Hom}_{\mathrm{gen}}(\mathcal{C}_S)|} \sum_{(u \xrightarrow{\ell} v) \in \mathrm{Hom}_{\mathrm{gen}}(\mathcal{C}_S)} \mathbb{I}\big[\mathcal{C}_T.\mathrm{reaches}(F(u), F(v), \ell) \vee \mathcal{C}_T.\mathrm{reaches}(F(u), F(v), \texttt{subClassOf})\big]. \tag{3}$$

Both the lattice identity functor and the wiki-to-lattice alignment functor are persisted as verifiable witnesses inside `lattice.json` alongside an induced OWL 2 TBox (`induced.ttl`).

3.3 Fail-Closed SPARQL Grounding (`leio-knowledge-core`)

All formal triples (code triples, OWL TBox cover axioms, and domain ontologies) are loaded into an in-process Oxigraph RDF store (`FormalStore`). When an agent queries `knowledge explain` or executes a SPARQL `SELECT`/`ASK` query Q, `sparql_with_store` enforces an *anchored-or-refuse* contract:

$$\mathrm{grounded}(Q, \mathcal{S}) = \begin{cases} \texttt{true}, & \text{if } |\mathrm{Bindings}(Q, \mathcal{S})| \geq 1, \\ \texttt{false} \text{ (Refusal Envelope)}, & \text{if } |\mathrm{Bindings}(Q, \mathcal{S})| = 0 \text{ or } Q \text{ errors.} \end{cases} \tag{4}$$

Empty bindings never fall back to generative guessing; the envelope returns `meta.grounded` = `false` with explicit refusal diagnostics.

4 Coverage-Calibrated Multi-Faceted Retrieval and ACE Playbook

When an agent invokes `leio_code_context(task)`, our goal is to return a compact, high-precision bundle containing the top-K relevant files ($K = 6$), focal symbols, verification commands, and architectural risk warnings in a single round-trip.

4.1 Why Naive Multi-Channel RRF Fails on Codebases

Suppose a task query q is tokenized into informative query stems $T(q) = \{t_1, \ldots, t_m\}$ (filtering stopwords and splitting snake/camelCase identifiers). A standard multi-layer retriever scores each file $f \in V_{\texttt{File}}$ across three channels—path lexical score $S_{\text{path}}(f, q)$, AST symbol score $S_{\text{sym}}(f, q)$, and operational config/env/Redis score $S_{\text{cfg}}(f, q)$—and combines their respective channel ranks $r_c(f) \in \{1, 2, \ldots\}$ via Reciprocal Rank Fusion [8]:

$$\text{RRF}_{\text{naive}}(f, q) = \sum_{c \in \{\text{path,sym,cfg}\}: S_c(f,q) > 0} \frac{\alpha_{\text{RRF}}}{k_0 + r_c(f)}, \qquad (k_0 = 60,\ \alpha_{\text{RRF}} = 25{,}000). \tag{5}$$

Proposition 1 (Single-Token Multi-Channel RRF Amplification Pathology)**.** *Under Eq. (5), each active channel awards up to* $\frac{\alpha_{RRF}}{k_0+1} \approx 409.8$ *rank points regardless of how many distinct query tokens* $t \in T(q)$ *matched in that channel. Consequently, a monolithic file* f_{god} *that matches a single generic token* $t_1 \in T(q)$ *in its path, one symbol name, and one environment variable string receives up to* $3 \times 409.8 \approx 1{,}229$ *RRF bonus points, outranking a focused implementation module* f^* *that matches all* $m \geq 4$ *distinct query tokens exclusively within its AST symbol definitions (*1×409.8 *RRF points).*

Our empirical ablation (§6) confirms Proposition 1 directly: adding naive 3-layer RRF to AST symbol scoring reduces Hit@1 from 68.0% to 52.0%!

4.2 Coverage-Calibrated Multi-Faceted RRF

We resolve this pathology in `src/context.rs` through five principled mechanisms:

1. Multi-Symbol Union Coverage & Density Damping. Rather than scoring a file solely by its single highest-scoring symbol ($\max_{s \in \text{sym}(f)} S(s, q)$), which penalizes cohesive modules whose responsibilities are distributed across helper functions (e.g., `deploy`, `verify_revision`, `rollback` in `delivery.rs`), we compute the union of distinct query tokens matched across all non-test symbols in f:

$$U_{\text{sym}}(f, q) = \bigcup_{s \in \text{sym}_{\text{prod}}(f)} \{t \in T(q) \mid \text{match}(t, s)\}, \tag{6}$$

and apply a BM25-style symbol-count damping factor to non-stem symbol bonuses:

$$\delta_{\text{density}}(f) = \frac{100}{\max(100,\ 100 + \max(0, |\text{sym}(f)| - 20) \times 3)}, \tag{7}$$

preventing 150-symbol server entrypoints from accumulating accidental token hits.

2. Morphological Prefix Matching and Module Entrypoint Stem Inheritance. Identifier tokens use morphological prefix matching ($\geq$ 5-char shared prefix when $\min(|a|, |b|)/\max(|a|, |b|) \geq 0.70$, aligning `config` $\leftrightarrow$ `configure` and `valid` $\leftrightarrow$ `validate`). Module entrypoints (`mod.rs`, `lib.rs`, `index.js`, `__init__.py`) inherit their parent directory's stem tokens.

3. Intent-Gated Config Channel and Single-Token Channel Collapse. Let $U_{\text{all}}(f, q) = U_{\text{path}}(f, q) \cup U_{\text{sym}}(f, q) \cup U_{\text{cfg}}(f, q)$ be the total distinct query token coverage of file f. When $|T(q)| \geq 3$:

- If $|U_{\text{all}}(f, q)| \leq 1$ and f has no exact multi-token needle match, we set $S_{\text{sym}}(f, q) \leftarrow 0$ and $S_{\text{cfg}}(f, q) \leftarrow 0$, collapsing multi-channel RRF amplification for single-token noise hits and applying a 0.35× raw score discount.

- If the query exhibits no configuration/environment/Redis intent ($\neg$wants_config(q)) and f is not a configuration file, we gate off $S_{\text{cfg}}(f, q) \leftarrow 0$.

4. Production vs. Test Role Prior and Coverage Bonus. Let $\mathbb{I}_{\text{test}}(f)$ indicate whether f resides in a test directory/filename (`tests/`, `*.test.*`, `*_test.rs`). When the query does not request tests ($\neg$wants_tests(q)), test files receive a role penalty (-700) while files matching $c = |U_{\text{all}}(f, q)| \geq 2$ distinct query tokens receive a superlinear coverage bonus:

$$B_{\text{cov}}(f, q) = (260c + 55c^2) \cdot (1 - 0.65\, \mathbb{I}_{\text{test}}(f) \cdot \mathbb{I}[\neg\text{wants_tests}(q)]). \tag{8}$$

5. ACE Evolutionary Co-Edit Playbook (`src/playbook.rs`). Coding tasks frequently require synchronized edits across a primary code module and an unindexed or weakly-matched partner (e.g., `mcp/tools.js` paired with `schema/leio-code-mcp-service-descriptor.json` or `docs/MCP-SURFACE-GAP.md`). LEIO CODE mines incremental commit history (`git log -name-only`, ignoring single-file commits and mega-sweeps > 20 files) to maintain a symmetric co-edit frequency matrix $C(f_a, f_b)$ and commit-revert outcome counters (helpfulness, harmfulness) in `.leio-code/playbook.json`. Given the top-1 ranked seed file f_1, its top co-edit partners f_p with $C(f_1, f_p) \geq 4$ receive a bounded multiplicative-floor injection:

$$\Delta_{\text{pb}}(f_p \mid f_1) = \min\Big(S(f_1) - 1,\ \max\big(\lfloor 0.75\, S(f_1) \rfloor,\ 260 \min(C(f_1, f_p), 8)\big)\Big), \tag{9}$$

lifting coupled manifest and documentation partners directly into the top-6 context bundle while strictly preserving $S(f_1) > S(f_p)$ (so Hit@1 is invariant).

5 Executable Architectural Drift Contracts (*Doctors*)

Prose instructions in `AGENTS.md` or `CLAUDE.md` cannot enforce repository invariants on their own. LEIO CODE compiles architectural invariants into 102 deterministic static analyzers (*doctors*) organized by repository profile. Representative doctors active on `leio-code` itself include:

- `self-contract` & `leio-release-coherence`: Verifies that CLI flags, MCP JSON schemas, descriptor manifests, and crate versions match across Rust, Node.js, and Python surfaces.
- `env-contract`: Cross-checks every $e \in V_{\texttt{EnvVar}}$ read in source code against declared environment schemas and deployment manifests.
- `redis-key-hygiene`: Flags Redis write operations lacking an associated TTL ('SET' without 'EX'/'PX' or subsequent 'EXPIRE').
- `orphan-files` & `import-boundary`: Detects unreferenced source modules and illegal cross-layer imports over $\mathcal{G}$.
- `vendored-crate-provenance` & `rust-toolchain-pin-coherence`: Enforces cryptographic provenance and toolchain version alignment across workspace crates.

To prevent warning fatigue, `Playbook` tracks (`fired_count`, `resolved_count`, `ignored_count`, `streak_commits`) per doctor warning across Git commits, automatically surfacing high-resolution vs. ignored rules via `leio-code doctor triage`.

Table 1: Main retrieval and localization efficiency results across the 25-task multi-category benchmark (`benchmarks/paper-evaluation-suite.json`) and 6-task coupled co-edit suite.

Retrieval Method	MRR	Hit@1	Hit@3	Hit@5	Partner@6	Mean Tokens	Tool Calls
1. `rg_lexical` (Ripgrep Baseline)	0.3193	20.0%	40.0%	52.0%	—	17,260.6	7.04
2. `path_only` (Path Lexical Only)	0.6747	52.0%	84.0%	88.0%	16.7%	1,019.1	3.04
3. `ast_symbol_only` (RepoMap / AST)	0.7367	68.0%	80.0%	84.0%	16.7%	1,013.6	2.96
4. `naive_rrf` (Uncalibrated 3-Layer RRF)	0.6847	52.0%	84.0%	92.0%	16.7%	1,018.1	3.00
5. `no_playbook` (Calibrated RRF Only)	**0.9800**	**96.0%**	**100.0%**	**100.0%**	16.7%	586.7	2.04
6. `full` (LEIO Code v2.6.5)	**0.9800**	**96.0%**	**100.0%**	**100.0%**	**100.0%**	**586.6**	**2.04**

6 Experimental Evaluation

We evaluate LEIO Code ('v2.6.5' release binary on macOS arm64) on the polyglot `leio-code` repository (349 indexed files across Rust, JavaScript/MJS, Python, JSON, and Bash; 1.7 MB index; 80 documentation files with 590 wiki sections; 9,726 RDF triples). All benchmarks and ablation scripts are reproducibly packaged in `scripts/benchmark_paper_ablations.py` and `scripts/evaluate_retrieval.py`.

6.1 25-Task Multi-Category Retrieval and Component Ablation Study

We construct a 25-task evaluation suite (`benchmarks/paper-evaluation-suite.json`) stratified evenly across 5 software engineering categories (5 tasks each): (1) *Symbol & Module Localization* (`symbol_module`), (2) *Infrastructure & Operational Topology* (`infra_topology`), (3) *Cross-Language & Protocol Wiring* (`cross_lang_protocol`), (4) *Formal Knowledge Graph & Lattice* (`knowledge_graph`), and (5) *Multi-Agent Harness & Evolution* (`multi_agent_harness`).

We compare six retrieval configurations:

1. **`rg_lexical` (Ripgrep Lexical Baseline):** Standard agent keyword search ranking files by distinct query token hit count across the repository.
2. **`path_only`:** LEIO Code using file path and directory stem scoring only ($S_{\text{sym}} = S_{\text{cfg}} = 0$, no RRF, no playbook).
3. **`ast_symbol_only` (RepoMap / LSP Baseline):** LEIO Code restricted to AST symbol definitions and call-graph proximity ($S_{\text{path}} = S_{\text{cfg}} = 0$, no RRF, no playbook).
4. **`naive_rrf` (Pre-Calibration 3-Layer RRF):** Uncalibrated 3-layer Reciprocal Rank Fusion over path, single-best symbol, and config channels without coverage calibration or role priors.
5. **`no_playbook`:** Full coverage-calibrated multi-faceted RRF with ACE co-edit playbook disabled.
6. **`full` (LEIO Code v2.6.5):** Complete coverage-calibrated multi-faceted RRF + ACE co-edit playbook.

Key Empirical Findings.

1. 3.07× **Higher MRR and** 29.4× **Token Reduction vs. Ripgrep:** As shown in Table 1, lexical `rg` achieves only MRR = 0.3193 and Hit@1 = 20.0% because benchmark artifacts, changelogs, and documentation files contain dense keyword matches. An agent following the `rg` ranking must read an average of 17,260.6 tokens across 7.04 tool calls to localize

Table 2: Per-category Mean Reciprocal Rank (MRR) and Top-1 Precision (Hit@1) across the five software engineering task categories.

	symbol_module		infra_topology		cross_lang		knowledge_graph		multi_agent	
Method	**MRR**	**H@1**	**MRR**	**H@1**	**MRR**	**H@1**	**MRR**	**H@1**	**MRR**	**H@1**
rg_lexical	0.450	20%	0.240	20%	0.000	0%	0.440	40%	0.467	20%
path_only	0.867	80%	0.740	60%	0.600	40%	0.467	20%	0.700	60%
ast_symbol_only	0.867	80%	0.400	40%	0.850	80%	0.867	80%	0.700	60%
naive_rrf	0.740	60%	0.550	40%	0.900	80%	0.667	40%	0.567	40%
full (LEIO)	**1.000**	**100%**	**1.000**	**100%**	**0.900**	**80%**	**1.000**	**100%**	**1.000**	**100%**

the primary target file, whereas LEIO Code delivers the target file at rank 1 on 24/25 tasks (96.0%) and within the top 2 on 25/25 tasks (100.0%) using 586.6 tokens and 2.04 calls (145.3 ms median release-binary latency). Also on the 7-task golden retrieval suite (`benchmarks/retrieval-agent-tasks.json`), LEIO Code achieves a perfect MRR = 1.0000 and Hit@1 = 100.0% (7/7).

2. **Complementarity of Operational Topology and AST Symbols:** Table 2 reveals why neither path matching nor AST symbol indexing suffices alone. On `infra_topology` tasks (Docker smoke scripts, Keycloak Compose wiring, systemd service installers), `ast_symbol_only` collapses to MRR = 0.400 (Hit@1 = 40%) because shell and container manifests lack AST symbol tables. Conversely, on `knowledge_graph` tasks (Oxigraph SPARQL refusal envelopes, category functor coherence), `path_only` collapses to MRR = 0.467 (Hit@1 = 20%) because file paths alone cannot disambiguate internal algebraic abstractions.

3. **Empirical Proof of the Naive RRF Pathology:** Combining channels via `naive_rrf` improves Hit@5 (84% → 92%) but *degrades* Hit@1 from 68.0% (`ast_symbol_only`) to 52.0%, exactly as predicted by Proposition 1. Adding coverage-calibrated RRF (`no_playbook` / `full`) eliminates single-token channel amplification and lifts Hit@1 by +44.0 percentage points (52.0% → 96.0%) and MRR from 0.6847 to 0.9800.

4. 6× **Gain in Coupled Co-Edit Partner Recall:** On multi-file maintenance tasks where a source module is historically co-edited with a JSON schema descriptor, Cargo manifest, or surface-gap specification, the ACE co-edit playbook increases partner recall in the 6-file context bundle (Partner@6) from 16.7% (1/6) to 100.0% (6/6) without disturbing top-1 seed ranking.

6.2 Formal Concept Lattice Induction Performance

Table 3 summarizes the formal concept lattice induced on the live repository via `leio-code export formal-context`. The native Rust bitset `In-Close` engine processes 8,343 repository objects, 4,061 structural attributes, and 67,642 incidences into 2,474 closed Galois concepts and an OWL 2 TBox in 593 ms, verifying 1.000 functorial coherence on both the cover-graph identity functor and the wiki heading hierarchy functor.

7 Conclusion

We presented LEIO Code, an infrastructure-aware repository intelligence engine that moves coding agents beyond syntactic call graphs. By unifying operational topology graphs, bitset-accelerated

Table 3: Formal Concept Analysis (FCA) Galois lattice and RDF/OWL knowledge base metrics on the `leio-code` repository.

Formal Artifact / Metric	**Measured Value**	**Verification / C**
Formal Objects $\|G\|$ (files, symbols, routes, services, wiki sections)	8,343	Indexed across
Formal Attributes $\|M\|$ (syntactic, operational, and domain properties)	4,061	Canonical sorted incide
Structural Incidences $\|I\| \subseteq G \times M$	67,642	SHA-256 incidence fing
Closed Formal Concepts $\|\mathfrak{B}(G, M, I)\|$	2,474	Pure Rust bitset I
Cover Category Identity Functor Coherence	1.000 (100%)	Verified by `evaluate_`
Wiki Heading Category $\rightarrow$ Lattice Functor Coherence	1.000 (100%)	Verified by `evaluate_`
Oxigraph Formal Store Triples (`leio-knowledge-core`)	9,726	Anchored-or-refuse SPARQL
End-to-End Lattice Induction + OWL TBox Export Latency	593 ms	Zero external Python dep

Formal Concept Analysis lattices, fail-closed Oxigraph SPARQL grounding, 102 executable drift contracts, and coverage-calibrated multi-faceted RRF with an evolving Git co-edit playbook, LEIO Code achieves 96.0% top-1 and 100.0% top-3 localization accuracy while reducing agent token consumption by 29.4×.

References

[1] J. Yang et al. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Proceedings of NeurIPS*, 2024.

[2] C. E. Jimenez et al. SWE-bench: Can language models resolve real-world GitHub issues? In *Proceedings of ICLR*, 2024.

[3] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang. Agentless: Demystifying LLM-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.

[4] P. Gauthier. Building a better repository map with Tree-sitter and PageRank. *Aider Technical Report*, 2024.

[5] Z. Chen et al. LocAgent: Graph-guided LLM agents for code localization. In *Proceedings of ACL*, 2025.

[6] S. Ouyang et al. RepoGraph: Enhancing AI software engineering with repository-level code graph. In *Proceedings of ICLR*, 2025.

[7] X. Liu et al. CodexGraph: Bridging large language models and code repositories via code graph databases. In *Proceedings of NAACL*, 2024.

[8] G. V. Cormack, C. L. A. Clarke, and S. Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of ACM SIGIR*, pages 758–759, 2009.

[9] B. Ganter and R. Wille. *Formal Concept Analysis: Mathematical Foundations*. Springer, 1999.

[10] S. Andrews. In-Close, a fast algorithm for computing formal concepts. In *International Conference on Conceptual Structures (ICCS)*, 2009.